alone to create an application that has a lower memory footprint, faster startup, and less surface area to worry about from a security perspective than a full ASP.NET Core application. .NET applications that use this approach are commonly called *worker services* or *workers.*

DEFINITION A *worker* is a .NET application that uses the generic `IHost` but doesn't include the ASP.NET Core libraries for handling HTTP requests. They are sometimes called *headless* services, as they don't expose a UI for you to interact with.

Workers are commonly used for running background tasks (`IHostedService` implementations) that don't require a UI. These tasks could be for running batch jobs, running tasks repeatedly on a schedule, or handling events using some sort of message bus. In the next section we'll create a worker for retrieving the latest exchange rates from a remote API instead of adding the background task to an ASP.NET Core application.

34.2.1 Creating a worker service from a template

In this section you'll see how to create a basic worker service from a template. Visual Studio includes a template for creating worker services: choose **File** > **New** > **Project** > **Worker Service**. You can create a similar template using the .NET command-line interface (CLI) by running `dotnet new worker`. The resulting template consists of two C# files:

- *Worker.cs*—This simple `BackgroundService` implementation writes to the log every second, as shown in listing 34.6. You can replace this class with your own `BackgroundService` implementation, such as the example from listing 34.5.